

Shakespeare (Hamlet) Next-Word Prediction with LSTM

Summary

In this project, I built a next-word prediction model on Shakespeare's Hamlet using N-gram sequences and an LSTM language model. I tokenized the play, created incremental N-gram sequences line by line, padded them to a fixed length, and trained a 2-layer LSTM with a softmax output over the full vocabulary.

Key facts from my run:

- Corpus: gutenber.shakespeare-hamlet.txt (lowercased)
- Vocabulary size: 4,818 tokens
- Max sequence length: 14 (inputs use 13 tokens, target is the 14th)
- Model: Embedding(4818×100) --> LSTM(150, seq) --> Dropout(0.5) --> LSTM(100) --> Dense(4818, softmax)
- Training: 100 epochs, batch size 16, optimizer Adam, loss categorical_crossentropy
- Final results: Train accuracy rose to ~0.63, but validation accuracy stayed ~0.04–0.07 and validation loss exploded to ~13.6 --> clear signs of overfitting.

1) Problem Formulation

Objective: Given a prefix of tokens ($x_1 \dots x_t$), predict the next word (x_{t+1}).

Learning setup: This is multi-class classification over the vocabulary (4,818 classes).

Metric: I used categorical accuracy, which requires an exact next-word match.

Why LSTM?

- Reviews or plays are sequential text where word order matters.
- LSTMs capture dependencies across time steps better than bag-of-words.
- I wanted this as a baseline before trying GRUs or Transformers.

2) Data Overview

Source: NLTK Gutenberg – shakespeare-hamlet.txt

Preprocessing steps I applied:

- Converted text to lowercase.
- Used Keras Tokenizer() (default filters removed punctuation).
- Did not set an OOV token, which means every rare word became its own class.

Vocabulary size: 4,818 unique words.

Notable feature: The vocabulary contains many archaic spellings (thinke, giue, teares)

A large vocabulary relative to the dataset makes the model harder to train. Many rare words appear only a handful of times, which encourages memorization.

3) Sequence Construction (N-grams per line)

I split the text into lines and built incremental N-grams within each line.

For example, from the line:

"to be or not to be"

I created the following sequences:

- [to, be]
- [to, be, or]
- [to, be, or, not]
- [to, be, or, not, to]
- [to, be, or, not, to, be]

For each sequence, the input is everything except the last token, and the label is the last token.

Counts:

- Total sequences: 25,732
- Maximum sequence length: 14
- Input shape: (25732, 13) after padding
- Target shape: one-hot vectors of size 4,818

Why padding?

Neural networks need fixed-size input. I padded shorter sequences to 13 tokens and used the 14th token as the target.

4) Train/Validation Split

I split the data randomly 80/20 into train and validation sets. Because the split was random across the entire play, the distributions are very similar. So the poor validation metrics aren't due to data imbalance, they're due to model issues.

5) Model Design

Architecture used:

- Embedding layer (input dim 4818, output dim 100)

- LSTM with 150 units (return sequences = True)
- Dropout(0.5)
- LSTM with 100 units (return sequences = False)
- Dense layer with 4818 units and softmax activation

How it works step by step (example):

Suppose input tokens are [w1, w2, ..., w13].

- The Embedding layer maps each word to a 100-dim vector.
- The first LSTM processes the sequence and outputs hidden states for each step.
- The second LSTM outputs a final hidden state (h13).
- The Dense+Softmax layer converts h13 into a probability distribution across 4,818 possible next words.

6) Training & Logs

Setup:

- Optimizer = Adam
- Loss = Categorical crossentropy
- Batch size = 16
- Epochs = 100
- No EarlyStopping or LR scheduling used

Observed behavior:

- Epoch 1 --> acc: 0.034, loss: 6.86 | val_acc: 0.035, val_loss: 6.75
- Epoch 10 --> acc: 0.091, loss: 5.20 | val_acc: 0.073, val_loss: 7.61
- Epoch 50 --> train accuracy keeps rising, val loss continues climbing
- Epoch 100 --> acc: 0.63, loss: 1.55 | val_acc: 0.05, val_loss: 13.6

Interpretation:

The training accuracy improves steadily, but validation accuracy stays close to random guessing (~0.02 baseline, since it's a 4.8k-way classification). Validation loss doesn't just plateau, it increases sharply, showing the model is memorizing training data without learning generalizable patterns.

7) Why Overfitting Happened

From the evidence, I think the overfitting happened for these reasons:

1. **Small dataset vs large model.**
Only ~26k sequences from one play versus probably more than 1million parameters.
The model has the capacity to memorize.
2. **Large vocabulary with rare words.**
Words like hecuba, giue, teares each have output neurons with very few examples.
This makes the softmax extremely sparse

3. **Short context windows (13 tokens).**
Shakespearean grammar would probably need longer context. The short windows make the model lean on memorization.
4. **Weak regularization and no training control.**
Only one Dropout layer. No EarlyStopping or learning-rate scheduling, and training ran for 100 epochs.
5. **Tokenizer defaults.**
Since I didn't set an OOV token, every rare word is treated as unique.
6. **Large softmax output.**
With 4,818 classes, exact-match accuracy is very harsh. Even a sensible guess that isn't word-for-word "correct" is probably marked wrong.

8) How It Could Be Fixed

A. Reduce vocabulary issues

- Limit `num_words` in the vocabulary and use an OOV token like `<OOV>`.
- This will map rare words to a common token and make training less sparse.

B. Improve context

- Increase `max_len` from 14 to 40 or 50, so the model sees more context.

C. Stronger regularization

- Use EarlyStopping (`patience=3–5`, restore best weights).
- Add `ReduceLROnPlateau` to adjust learning rate when `val_loss` stalls.
- Try smaller LSTMs (e.g. GRU(128)) with dropout inside (`dropout=0.2`, `recurrent_dropout=0.2`).

9) Inference Pipeline

The inference pipeline works as follows:

1. Tokenize the input sentence.
2. Pad to length 13 (since training inputs were 13).
3. Use `model.predict` to get a probability distribution.
4. Sort and map back to words.
5. Show the top-k predictions.